Redroid Feasibility for the Lava §6.X / §6.AH Emulator Gate

- Date: 2026-07-02
- Host: `nezha`, kernel `6.12.61-6.12-alt1`, `x86_64` (ALT Linux)
- Runtime: rootless `podman 5.7.1` (no docker installed)
- Investigator: read-only feasibility subagent (no source edits, no gradle, no container mutations)
- Redroid upstream: https://github.com/remote-android/redroid-doc

Verdict (TL;DR)

NO-GO on this host right now — but **GO as an adoptable §6.X/§6.AH runner on a properly-provisioned Linux gate host.**

Single biggest blocker: **this host's kernel has `CONFIG_ANDROID_BINDER_IPC` NOT set and ships zero loadable binder/ashmem modules.** Redroid physically cannot start without the binder driver. Providing it requires host-level root actions (`apt install linux-modules-extra-$(uname -r)` + `modprobe binder_linux ashmem_linux`) which §6.U forbids inside the repo — and it is unconfirmed that this ALT Linux kernel even packages those modules.

The upside worth capturing: Redroid is the **strongest §6.X/§6.AH candidate found to date** because it needs **no `/dev/kvm`, no QEMU, no TCG**, and it maps almost 1:1 onto the existing `emulator.Emulator` interface. Once binder is provisioned once on a Linux gate host, adoption is a small, well-bounded Go addition.

---

1. Host kernel prerequisites — ABSENT (hard blocker)

Redroid runs a real Android userspace directly on the host Linux kernel, so it requires the Android kernel drivers `binder_linux` (+ `binderfs`) and `ashmem_linux` (or `androidboot.use_memfd`) to be present and loaded on the **host**. Probes of this host:

Probe	Command	Result
Kernel config	`zcat /proc/config.gz | grep -iE 'BINDER|ASHMEM|CONFIG_ANDROID'`	**`# CONFIG_ANDROID_BINDER_IPC is not set`** — and that is the ONLY match. No `BINDERFS`, no `ASHMEM`, no `CONFIG_ANDROID` symbol at all.
binder device nodes	`ls -la /dev/binder* /dev/*binder*`	NONE in /dev
binderfs mount	`ls /dev/binderfs` + `mount | grep binder`	absent; no binder mounts
loaded modules	`lsmod | grep -iE 'binder|ashmem'`	none loaded
module in sysfs	`ls /sys/module/binder_linux`	absent
module files on disk	`find /lib/modules/6.12.61-6.12-alt1 -iname '*binder*' -o -iname '*ashmem*'`	**no binder/ashmem module files found**
modinfo	`modinfo binder_linux` / `modinfo ashmem_linux`	not found (empty)
android drivers dir	`ls /lib/modules/$(uname -r)/kernel/drivers/android`	does not exist
modprobe present	`command -v modprobe`	**not in PATH**

Conclusion: binder is not merely unloaded — it is **not built into the kernel AND has no loadable module available on disk.** `binderfs` and `ashmem` depend on `CONFIG_ANDROID_BINDER_IPC`, so they are necessarily absent too. This is the decisive blocker.

What would be needed to unblock (and the §6.U conflict):

- Upstream redroid recipe: `apt install linux-modules-extra-$(uname -r)` then `modprobe binder_linux devices="binder,hwbinder,vndbinder"` and `modprobe ashmem_linux`.
- On ALT Linux (not Debian/apt), the equivalent kernel-modules-extra package would have to exist and be installed — **UNCONFIRMED that ALT ships these modules for `6.12.61-6.12-alt1`; it must be verified against ALT's `kernel-modules-*` repo.** If ALT does not package them, the only path is a custom kernel rebuild with `CONFIG_ANDROID_BINDER_IPC=y/=m` + `CONFIG_ANDROID_BINDERFS=y` + `CONFIG_ASHMEM` (or a memfd-capable redroid image).
- Both installing packages and `modprobe`-ing modules require **root**. Per **§6.U (no sudo/su in any tracked artifact)**, a committed Lava/Containers script may NOT do this. The module load must be a one-time **host-provisioning action performed by the operator outside the repo** (e.g. persisted via `/etc/modules-load.d/redroid.conf` on the gate host). Once the modules are loaded and `/dev/binderfs` is mountable, a rootless container consuming an already-present binder device does not itself need root — so the §6.U conflict is confined to host provisioning, not to the runner code.

---

2. Rootless podman compatibility — UNRESOLVED, needs a real spike (second blocker)

Redroid's documented quick-start (quoted verbatim from redroid-doc):

```
docker run -itd --rm --privileged \
    --pull always \
    -v ~/data:/data \
    -p 5555:5555 \
    redroid/redroid:12.0.0_64only-latest
```

- It uses **`--privileged`**. Lava's posture is **§6.V rootless podman only + no `--privileged`** (see root CLAUDE.md §6.V clause 2: "No privileged containers, no `--privileged` flag").
- The redroid docs contain **no mention of rootless podman and no non-privileged recipe** (WebFetch of the README confirmed this).
- Community practice (outside the official docs — treat as UNCONFIRMED until spiked here) narrows `--privileged` down to explicit device access, e.g. bind-mounting `/dev/binder*` / `/dev/ashmem` (or a per-container `binderfs` instance) plus an unconfined AppArmor/seccomp profile. Whether that fully-narrowed form works under **rootless** podman (where the

container user maps to an unprivileged subuid — this host has `milosvasic:100000:65536` in /etc/subuid/subgid) is **not established** and is exactly the kind of device-cgroup + userns interaction that broke KVM access in the existing `containerized.go` (see its `--userns=keep-id` RC1 fix).

Assessment: a rootless-compatible, non-`--privileged` redroid configuration is *plausible* (binder is a char device; if the host ACL grants the invoking user access and the device is passed with `--device`, the model is analogous to the existing `--device /dev/kvm` + `--userns=keep-id` path). But it is **undocumented upstream and unproven here** — it requires a real rootless spike **after** binder modules are present. Until that spike passes, treat rootless-without-privileged as an open risk, not a given.

---

3. ADB + instrumentation + recording fit — EXCELLENT

- **ADB:** redroid exposes adbd on TCP `5555`; you connect with `adb connect localhost:5555` (docs verbatim: "adb connect localhost:5555", substitute IP for remote). This is exactly the connection model `Containerized.WaitForBoot` already uses (`adb connect localhost:<port>` → `adb -s localhost:<port> ...`).
- **Instrumentation:** the existing flow is `adb install -r <apk>` → gradle `:<module>:connectedDebugAndroidTest` with `ANDROID_SERIAL=localhost:<port>`. Redroid is a full Android with a normal adbd, so `adb install` + `am instrument` (which is what connectedDebugAndroidTest drives) work unchanged.
- **Simplification vs. QEMU/containerized:** redroid needs **none** of the containerized path's boot ceremony — no AVD system-image fetch, no `/dev/kvm`, no baked adb-keypair `authorizeADB()` dance, no `socat 5575→5555` bridge, no `sys.boot_completed` KVM-accel timing. adbd is directly reachable on the published port; boot-completed polling still applies but is faster.
- **Recording:** docs confirm `scrcpy -s localhost:5555` works against redroid; `scrcpy --record file.mp4` therefore satisfies the §11.4.128 always-on device-recording debt for redroid targets.

---

4. Current Containers runner surface + where redroid plugs in

The interface a redroid runner implements (`submodules/containers/pkg/emulator/types.go:309-334`):

```
type Emulator interface {
    Boot(ctx, avd, coldBoot) (BootResult, error)
    WaitForBoot(ctx, port, timeout) (time.Duration, error)
    Install(ctx, port, apkPath) error
    RunInstrumentation(ctx, port, testClass, timeout) (output
        string, passed bool, err error)
    Teardown(ctx, port) error
}
```

`MatrixRunner`/`AndroidMatrixRunner` orchestrates any `Emulator` through Boot→WaitForBoot→Install→RunInstrumentation→Teardown and writes the per-AVD attestation. **Nothing in the matrix runner is QEMU/KVM-specific** — it is fully polymorphic over `Emulator`. A `RedroidEmulator` slots in with zero matrix-runner changes.

Concrete Go changes (new file `pkg/emulator/redroid.go`)

A `RedroidEmulator` closely mirrors `Containerized` (`containerized.go`) but drops the KVM/AVD-image/keypair machinery:

- `type RedroidEmulator struct { runtimeBinary, image string; executor CommandExecutor; containerName string; hostADBPort int; adbBinaryPath, gradleBinary, gradleModule string }`
- `NewRedroid(cfg RedroidConfig) (*RedroidEmulator, error)` — validates `RuntimeBinary` + `Image` (reuse the ContainerizedConfig fail-loud pattern).
- `Boot()` — `podman run -d --rm --name lava-redroid-<avd>-<ms> -v <data>:/data -p <hostPort>:5555 [device/userns flags — NOT --privileged] <image>`. `buildRedroidRunArgs()` is the unit-testable seam (mirror `buildContainerRunArgs`), with a **falsifiability rehearsal proving `--privileged` is NEVER emitted** (anti-bluff guard against silently regressing to the docs' quick-start).
- `WaitForBoot()` — `adb connect localhost:<port>` then poll `getprop sys.boot_completed == 1`. No `authorizeADB()` needed.
- `Install()` / `RunInstrumentation()` / `Teardown()` — reuse the identical implementations from `containerized.go` (they are already runtime-agnostic; `RunInstrumentation` uses the shared `gradleConnectedTestArgs`).
- `var _ Emulator = (*RedroidEmulator)(nil)` compile-time check.
- Add a `Pre-flight` that stat-checks `/dev/binderfs` (or `/dev/binder`) and **fails loud with an honest §6.J error** ("redroid requires host binder; CONFIG_ANDROID_BINDER_IPC absent — see research 2026-07-02") when binder is missing, rather than emitting a container that never boots.

CLI wiring (`cmd/emulator-matrix/main.go`)

- Extend `--runner` help + `ResolveRunner` (`pkg/emulator/accel.go`) to accept `redroid` as an explicit runner value. On Linux, `redroid` is gate-eligible **iff** host binder is present.

- Reuse the existing `--container-image` + `--container-runtime` flags (redroid uses `--container-image redroid/redroid:14.0.0_64only-latest`).
- Add a `redroid` branch to the emulator-construction switch (alongside `RunnerContainerized`) building `NewRedroid(...)`.

Lava-side glue (`scripts/run-challenge-matrix.sh`)

- Add `--runner=redroid` passthrough (the script already forwards `--runner`; today it hard-forwards `auto`). Add a Linux pre-flight that checks `[ -e /dev/binderfs ] || [ -e /dev/binder ]` and exits 2 with an honest "binder absent — provision per research 2026-07-02" message (mirrors the existing KVM pre-flight `exit 2`).

Image manifest (`tools/lava-containers/vm-images.json`)

Nuance: `vm-images.json` is a **`pkg/cache.Manifest`** (fields `url`/`sha256`/`size`/`format`) for **downloadable qcow2 / android-system-image blobs** fetched + SHA-verified by `pkg/cache.Store.Get`. Redroid images are **OCI images pulled by podman**, not cache blobs — they do not fit the existing schema cleanly. Two honest options:

1. Add a new `"format": "oci-image"` entry (e.g. `id: redroid-android14-x86_64`, `url: docker.io/redroid/redroid:14.0.0_64only-latest`, sha via image digest) and teach the redroid path to `podman pull` by digest rather than routing through `pkg/cache` — keeps one manifest, honest about the different fetch mechanism.
2. Keep redroid image refs in a separate small `redroid-images.json` so the SHA-verified qcow2 cache manifest stays pure. Recommended, lower blast radius.

---

5. Phased adoption plan (if/when GO)

Blockers to clear, in order:

1. **[HOST, operator, §6.U-external]** Confirm ALT Linux packages `binder_linux`+`ashmem_linux` for `6.12.61-6.12-alt1` (or a memfd-capable path); if so, operator loads them once on the Linux gate host via `/etc/modules-load.d/` + mounts `binderfs`. If ALT does not package them → custom kernel build with `CONFIG_ANDROID_BINDER_IPC` + `CONFIG_ANDROID_BINDERFS` + `CONFIG_ASHMEM`. **This is the single biggest blocker.**
2. **[SPIKE, rootless]** With binder present, prove a **rootless podman + non-`--privileged`** redroid boot (narrowed `--device`/`--userns=keep-id`/security-opt). If it needs `--privileged`, it

is **§6.V-non-compliant** and adoption stops until a non-privileged recipe is found.

Then, once both pass:

- Phase A — `pkg/emulator/redroid.go` (`RedroidEmulator` + `buildRedroidRunArgs` + binder pre-flight) with falsifiability-rehearsed unit tests (no-`--privileged` guard; adb-connect/boot-poll seam via the fake `CommandExecutor`). Upstream to Containers first (Decoupled Reusable Architecture rule).
- Phase B — `cmd/emulator-matrix --runner=redroid` + `ResolveRunner`/`accel.go` gate-eligibility (Linux+binder). Bump Containers pin in Lava.
- Phase C — Lava glue: `scripts/run-challenge-matrix.sh --runner=redroid` + binder pre-flight; redroid image entry (separate `redroid-images.json` recommended).
- Phase D — one real gate run on the provisioned Linux host producing a genuine per-AVD attestation (§6.I/§6.AE), + wire scrcpy `--record` for §11.4.128 recording. Only then does redroid become a gate-eligible runner.

Why this matters despite the NO-GO

Redroid is the first candidate that could close **§6.X-debt / §6.AH-debt without `/dev/kvm` and without QEMU/TCG**: it runs native Android on the host kernel, fits the existing `Emulator` interface almost verbatim (and is simpler than the KVM containerized path — no system-image fetch, no adb-keypair authorize, no socat bridge), and supports Android 8.1→16 on amd64+arm64. The entire adoption reduces to two provisioning preconditions (host binder modules; a proven rootless non-privileged recipe) plus a small, bounded Go addition. Recommend confirming ALT's binder-module availability and running the rootless spike on the intended Linux gate host as the next step.